

## **Q.1) Explain Risk and its types? Explain the steps involved in setting up or generating RMMM plan.**

**Ans: Risk:**

1. A risk is any uncertain event or condition that may negatively affect a software project's objectives such as cost, schedule, quality, or performance.
2. Risk management involves identifying, analysing, planning, and controlling potential risks to minimize their impact on the project.
3. The goal is to predict possible problems before they occur and prepare strategies to handle them effectively.
4. There are several types of risks that can occur in a project:

**i) Project Risk:** These risks affect project schedule, budgets, resources, or overall management.

**Examples:** Unrealistic deadlines, lack of skilled staff, poor communication.

**ii) Technical Risk:** These arise from design, development, implementation, or technology-related issues.

**Examples:** Use of new technology, unclear technical requirements, software integration failure.

**iii) Business Risk:** These are external risks that can affect the overall success of the product or organization.

**Examples:** Change in market demand, customer dissatisfaction, financial loss.

**iv) Operational Risk:** These occur during the operation or maintenance phase.

**Examples:** System downtime, lack of user training, operational failure.

**v) Schedule Risk:** These risks occur when project activities take longer than expected.

**Examples:** Delay in module delivery, unrealistic time estimation.

### **5. Steps involved in Setting up or Generating an RMMM Plan:**

**i) Risk Mitigation:**

- This step focuses on minimizing the chances of risk occurrence or reducing its impact.

- It includes preventive actions such as using proven technologies, assigning experienced team members, conducting regular reviews, and maintaining backup plans.
- The goal is to avoid or lessen the effect of potential project risks before they occur.

## **ii) Risk Monitoring:**

- In this step, all identified risks are continuously tracked throughout the software project.
- The project team regularly checks the risk indicators, evaluates their status, and updates the risk list.
- Monitoring helps in early detection of new risks or changes in existing risks so that corrective measures can be applied on time.

## **iii) Risk Management:**

- This step defines the actions to be taken after a risk actually occurs.
  - It ensures that the project can continue with minimal disruption by following pre-decided recovery strategies.
  - **Example:** Using backup systems, alternate schedules, or additional resources to recover from failure.
  - The main objective is to control the damage and bring the project back on track quickly.
-

## Q.2) Explain COCOMO model in detail.

### Ans: Constructive Cost Model (COCOMO):

- COCOMO is one of the most widely used software estimation models in the world.
- This model is developed in 1981 by Barry Boehm to give estimation of number of man-months it will take to develop a software product.
- COCOMO predicts the efforts and schedule of software product based on size of software.
- COCOMO has three different models that reflect complexity

1. **Basic Model**
2. **Intermediate Model**
3. **Detailed Model**

- Similarly, there are three classes of software projects.

1. **Organic mode:** In this mode, relatively simple, small software projects with a small team are handled. Such team should have good application experience to less rigid requirements.
2. **Semi-detached projects:** In this class intermediate project in which team with mixed experience level are handled. Such project may have mix of rigid and less than rigid requirements.
3. **Embedded projects:** In this class, project with tight hardware, software and operational constraints are handled.

- **Each Model in detail**

### 1. Basic Model:

The basic COCOMO model estimate the software development effort using only Lines of code.

Various equations in this model are

$$E = a_b KLOC_b^d$$
$$D = c_b E_b^d$$

Where, **E** is the effort applied in person-months,  
**D** is the development time in chronological months and  
**KLOC** is the estimated number of delivered lines of code for the project

## **2. Intermediate Model**

This is extension of COCOMO model.

This estimation model makes use of set of “Cost Driver Attributes” to compute the cost of software.

### ***I. Product attributes***

- a. required software reliability
- b. size of application data base
- c. complexity of the product

### ***II. Hardware attributes***

- a. run-time performance constraints
- b. memory constraints
- c. volatility of the virtual machine environment
- d. required turnaround time

### ***III. Personnel attributes***

- a. analyst capability
- b. software engineer capability
- c. applications experience
- d. virtual machine experience
- e. programming language experience

### ***IV. Project attributes***

- a. use of software tools
- b. application of software engineering methods
- c. required development schedule

Each of the 15 attributes is rated on a 6 point scale that ranges from “very low” to “extra high” (in importance or value).

The intermediate COCOMO model takes the form

$$E = a_i KLOC_i^b \times EAF$$

Where, **E** is the effort applied in person-months and **KLOC** is the estimated number of delivered lines of code for the project

#### 4. Detailed COCOMO Model

The detailed model uses the same equation for estimation as the intermediate Model. But detailed model can estimate the effort (E), duration (D), and person (P) of each of development phases, subsystem and models.

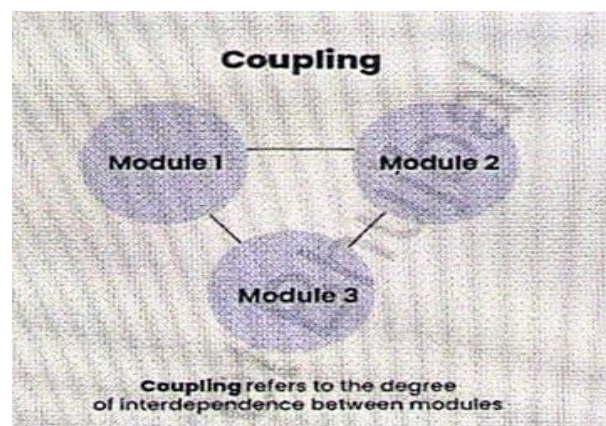
Sure! Here is the **word-to-word** transcription of the full text from all the images you uploaded (about **Coupling and Cohesion**):

---

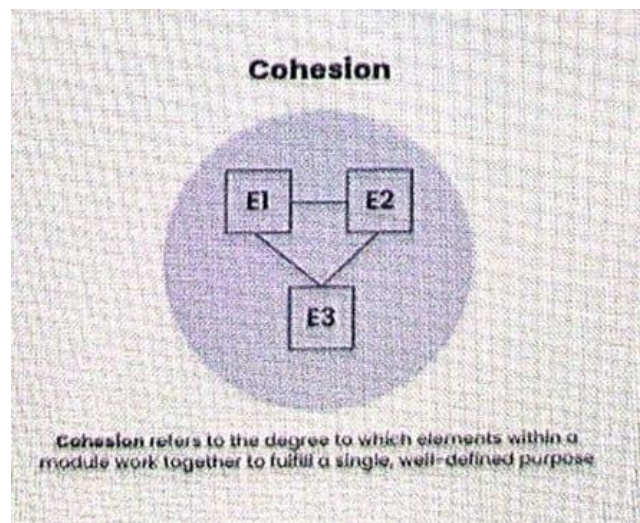
### Q.4) Explain coupling and cohesion. Explain different types of coupling and cohesion.

**Ans:**

**Coupling** refers to the degree of interdependence between software modules. High coupling means that modules are closely connected and changes in one module may affect other modules. Low coupling means that modules are independent, and changes in one module have little impact on other modules

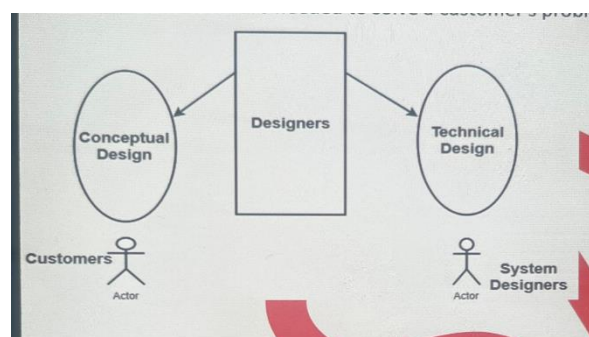


**Cohesion** refers to the degree to which elements within a module work together to fulfil a single, well-defined purpose. High cohesion means that elements are closely related and focused on a single purpose, while low cohesion means that elements are loosely related and serve multiple purposes.

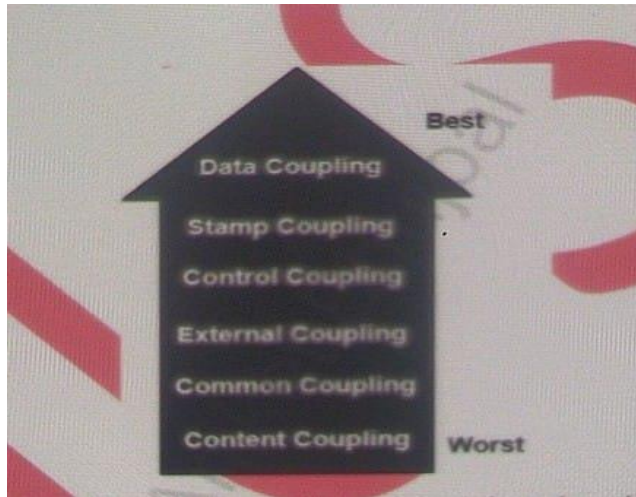


Both coupling and cohesion are important factors in determining the maintainability, scalability, and reliability of a software system. High coupling and low cohesion can make a system difficult to change and test, while low coupling and high cohesion make a system easier to maintain and improve.

Basically, design is a two-part iterative process. The first part is Conceptual Design which tells the customer what the system will do. Second is Technical Design which allows the system builders to understand the actual hardware and software needed to solve a customer's problem.



## Types of Coupling



### **Data Coupling:**

If the dependency between the modules is based on the fact that they communicate by passing only data, then the modules are said to be data coupled. In data coupling, the components are independent of each other and communicate through data. Module communications don't contain tramp data.

**Example:** customer billing system.

### **Stamp Coupling:**

In stamp coupling, the complete data structure is passed from one module to another module. Therefore, it involves tramp data. It may be necessary due to efficiency factors. This choice was made by the insightful designer, not a lazy programmer.

### **Control Coupling:**

If the modules communicate by passing control information, then they are said to be control coupled. It can be bad if parameters indicate completely different behaviour and good if parameters allow factoring and reusing.

**Example:** function that takes comparison function as an argument.

### **External Coupling:**

In external coupling, the modules depend on other modules, external to the software being developed or to a particular type of hardware.

**Example:** protocol, external file, device format, etc.

**Common Coupling:**

The modules have shared data such as global data structures. The changes in global data mean tracing back to all modules which access that data to evaluate the effect of the change. So it has got disadvantages like difficulty in reusing modules, reduced ability to control data accesses, and reduced maintainability.

**Content Coupling:**

In a content coupling, one module can modify the data of another module, or control flow is passed from one module to the other module. This is the worst form of coupling and should be avoided.

**Temporal Coupling:**

Temporal coupling occurs when two modules depend on the timing or order of events, such as one module needing to execute before another. This type of coupling can result in design issues and difficulties in testing and maintenance.

**Sequential Coupling:**

Sequential coupling occurs when the output of one module is used as the input of another module, creating a chain or sequence of dependencies. This type of coupling can be difficult to maintain and modify.

**Communicational Coupling:**

Communicational coupling occurs when two or more modules share a common communication mechanism, such as a shared message queue or database. This type of coupling can lead to performance issues and difficulty in debugging.

**Functional Coupling:**

Functional coupling occurs when two modules depend on each other's functionality, such as one module calling a function from another module. This type of coupling can result in tightly-coupled code that is difficult to modify and maintain.

**Data-Structured Coupling:**

Data-structured coupling occurs when two or more modules share a common data structure, such as a database table or data file. This type of coupling can lead to difficulty in maintaining the integrity of the data structure and can result in performance issues.

**Interaction Coupling:**

Interaction coupling occurs when one class is dependent on class invoking methods of other classes. Like with functions, the best form of coupling here is if methods



directly access internal parts of other methods. Coupling is lowest if methods communicate directly through parameters.

### **Component Coupling:**

Component coupling refers to interaction between two classes where a class has variables of the other class. Three clear situations exist as to how this can happen. A class C can be component coupled with another class C1, if C has an instance variable of type C1, or C has a method whose parameter is of type C1, or if C has a method which has a local variable of type C1. It should be clear that whenever there is component coupling, there is likely to be interaction coupling.

## **Types of Cohesion**

### **Functional Cohesion:**

Every essential element for a single computation is contained in the component. A functional cohesion performs the task and functions. It is an ideal situation.

### **Sequential Cohesion:**

An element outputs some data that becomes the input for another element, i.e., data flow between the parts. It occurs naturally in functional programming languages.

### **Communicational Cohesion:**

Two elements operate on the same input data or contribute towards the same output data.

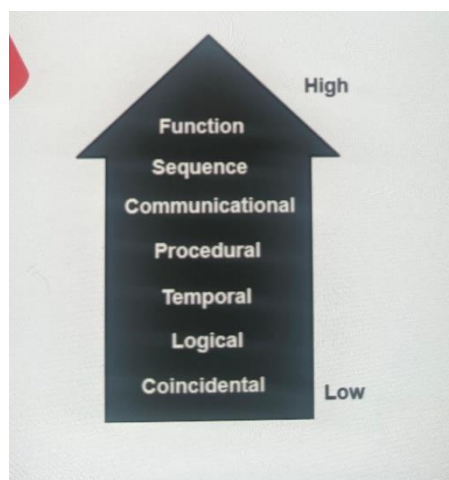
**Example:** update record in the database and send it to the printer.

### **Procedural Cohesion:**

Elements of procedural cohesion ensure the order of execution. Actions are still weakly connected and unlikely to be reusable.

**Example:** calculate student GPA, print student record, calculate cumulative GPA, print cumulative GPA.

### **Temporal Cohesion:**



The elements are related by their timing involved. A module connected with temporal cohesion all the tasks must be executed in the same time span. This cohesion contains the code for initializing all the parts of the system. Lots of different activities occur, all at unit time.

**Logical Cohesion:**

The elements are logically related and not functionally.

**Example:** A component reads inputs from tape, disk, and network. All the code for these functions is in the same component. Operations are related, but the functions are significantly different.

**Coincidental Cohesion:**

The elements are not related (unrelated). The elements have no conceptual relationship other than location in source code. It is accidental and the worst form of cohesion.

**Example:** print next line and reverse the characters of a string in a single component.

**Procedural Cohesion:**

This type of cohesion occurs when elements or tasks are grouped together in a module based on their sequence of execution, such as a module that performs a set of related procedures in a specific order. Procedural cohesion can be found in structured programming languages.

**Communicational Cohesion:**

Communicational cohesion occurs when elements or tasks are grouped together in a module based on their interactions with each other, such as a module that handles all interactions with a specific external system or module. This type of cohesion can be found in object-oriented programming languages.

**Temporal Cohesion:**

Temporal cohesion occurs when elements or tasks are grouped together in a module based on their timing or frequency of execution, such as a module that handles all periodic or scheduled tasks in a system. Temporal cohesion is commonly used in real-time and embedded systems.

**Informational Cohesion:**

Informational cohesion occurs when elements or tasks are grouped together in a module based on their relationship to a specific data structure or object, such as a module that operates on a specific data type or object. Informational cohesion is commonly used in object-oriented programming.

**Functional Cohesion:**

This type of cohesion occurs when all elements or tasks in a module contribute to a single well-defined function or purpose, and there is little or no coupling between the elements. Functional cohesion is considered the most desirable type of cohesion as it leads to more maintainable and reusable code.

**Layer Cohesion:**

Layer cohesion occurs when elements or tasks in a module are grouped together based on their level of abstraction or responsibility, such as a module that handles only low-level hardware interactions or a module that handles only high-level business logic. Layer cohesion is commonly used in large-scale software systems to organize code into manageable layers.

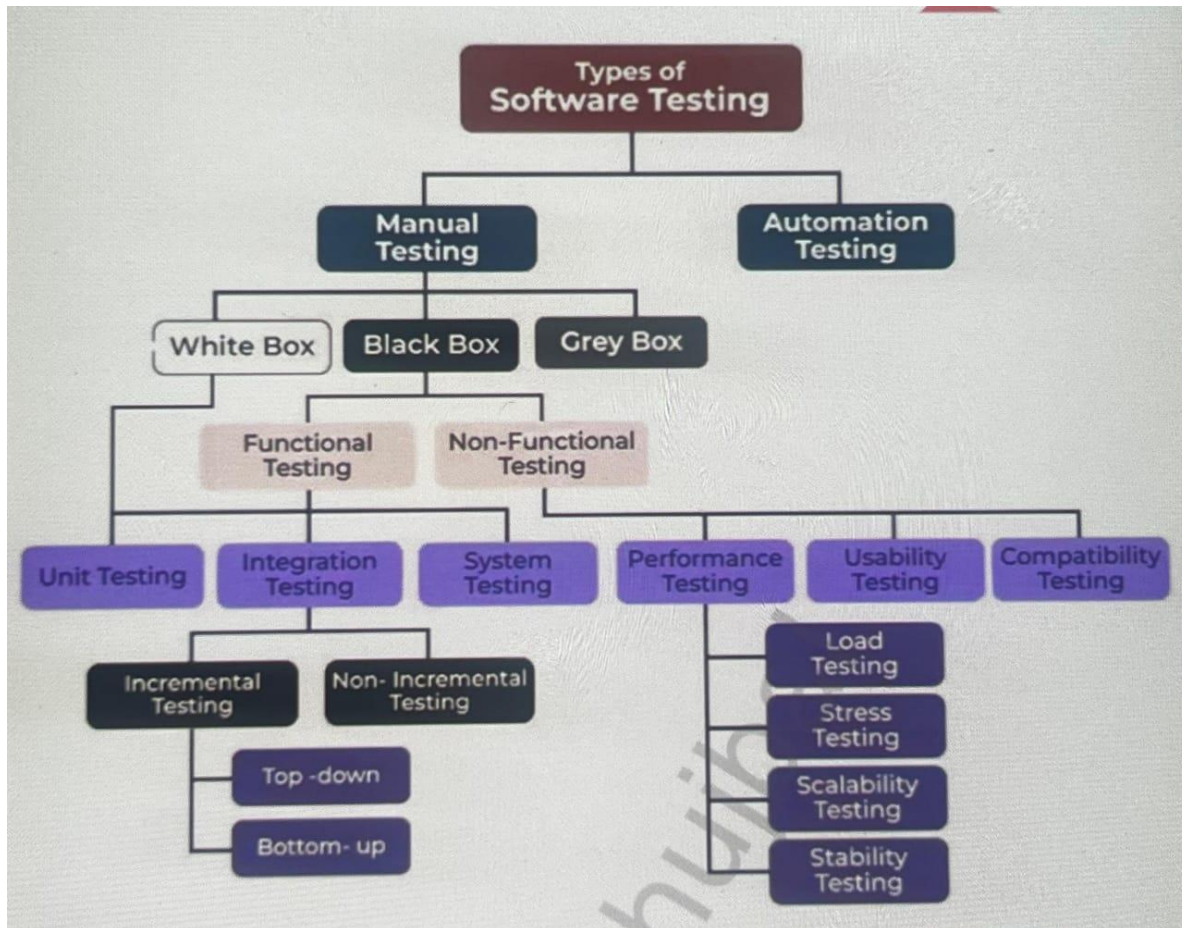
---

**Q.5) Explain different types of software testing with its principle.**

**Ans:**

**Different Types of Software Testing**

Here are the Types of Software Testing mainly categorized into the two domain, which are below.



## 1. Manual Testing

Manual Testing is a technique to test the software that is carried out using the functions and features of an application. Which means manual testing will be check the defect manually with trying one by one function is working as expected.

### Advantages of Manual Testing:

#### i) Fast and accurate visual feedback:

It detects almost every bug in the software application and is used to test the dynamically changing GUI designs like layout, text, etc.

**ii) Less expensive:**

It is less expensive as it does not require any high-level skill or a specific type of tool.

**iii) No coding is required:**

No programming knowledge is required while using the black box testing method. It is easy to learn for the new testers.

**iv) Efficient for unplanned changes:**

Manual testing is suitable in case of unplanned changes to the application, as it can be adopted easily.

## **2. Automation Testing**

Automated Testing is a technique where the Tester writes scripts independently and uses suitable Software or Automation Tools to test the software.

It is an Automation Process of a Manual Process. It allows for executing repetitive tasks without the use of a Manual Tester.

**Advantages of Automation Testing:**

**i) Simplifies Test Case Execution:**

Automation testing can be left virtually unattended and thus it allows monitoring of the results at the end of the process. Thus, simplifying the overall test execution and increasing the efficiency of the application.

**ii) Improves Reliability of Tests:**

Automation testing ensures that there is equal focus on all the areas of the testing, thus ensuring the best quality end product.

**iii) Increases amount of test coverage:**

Using automation testing, more test cases can be created and executed for the application under test. Thus, resulting in higher test coverage and the detection of more bugs. This allows for the testing of more complex applications and more features can be tested.

**iv) Minimizing Human Interaction:**

In automation testing, everything is automated from test case creation to execution

thus there are no chances for human error due to neglect.  
This reduces the necessity for fixing glitches in the post-release phase.

## Principles of Software Testing

Software Testing always aligns with the Customer's Requirement, which they want.

Software testing is an important process that is used for the enhancement of the Software Quality and Reliability of the application.

It is important to understand the key principle of software testing, which guides you throughout the process of Software Development.

These principles will be helpful for the tester to identify the software issue earlier and verify the Software meets to the expectations.

1. **Testing shows the presence of defects.**
  2. **Exhaustive Testing is not possible**, you cannot test input or scenario. Instead, focus on the most critical and risk-prone areas.
  3. **Early testing saves time and cost.**
  4. **Defect Clustering**, most defects are usually found in a small number of modules. Focus more testing on these high-risk areas.
  5. **Pesticide Paradox**, running the same tests repeatedly won't find new bugs. You must regularly update and improve your test cases.
  6. **Testing is Context-Dependent**, the testing approach varies based on the type of application (web, mobile, embedded, etc.).
  7. **Absence of Errors Fallacy**, even if your software has no bugs, it may still fail if it doesn't meet the user's needs or expectations.
- 

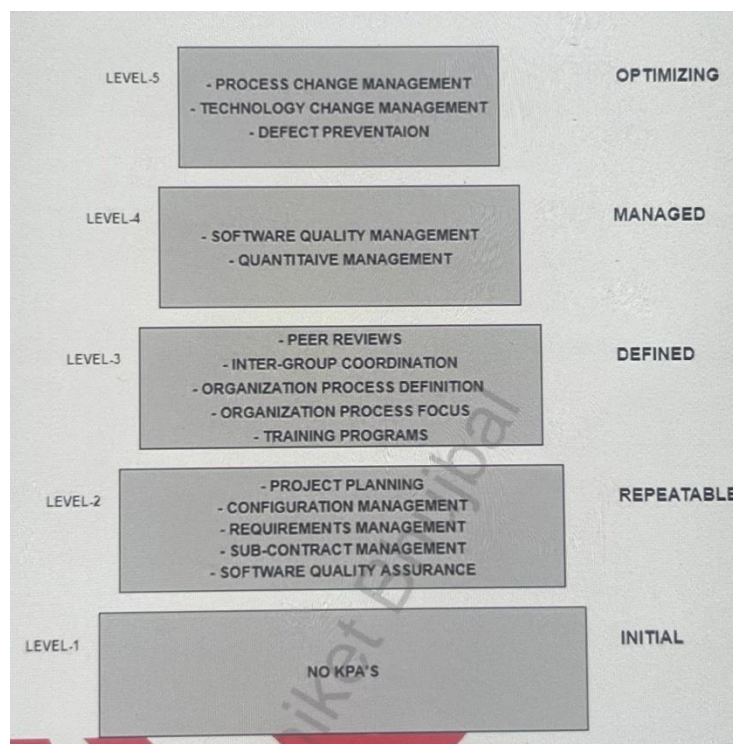
### Q.6) What is Capability Maturity Model (CMM)? Explain different CMM levels.

**Ans:**

i) **Capability Maturity Model (CMM)** was developed by the Software Engineering Institute (SEI) at Carnegie Mellon University in 1987.

- ii) It is not a software process model. It is a framework that is used to analyze the approach and techniques followed by any organization to develop software products.
- iii) It also provides guidelines to enhance further the maturity of the process used to develop those software products.
- iv) It is based on profound feedback and development practices adopted by the most successful organizations worldwide.
- v) This model describes a strategy for software process improvement that should be followed by moving through 5 different levels. Each level of maturity shows a process capability level.
- vi) All the levels except level 1 are further described by Key Process Areas (KPA).

## Levels of Capability Maturity Model (CMM)



There are 5 levels of Capability Maturity Models. We will discuss each one of them in detail.



## Level 1: Initial

- i) No KPIs defined.
- ii) Processes followed are Adhoc and immature and are not well defined.
- iii) Unstable environment for software development.
- iv) No basis for predicting product quality, time for completion, etc.
- v) Limited project management capabilities, such as no systematic tracking of schedules, budgets, or progress.
- vi) We have limited communication and coordination among team members and stakeholders.
- vii) No formal training or orientation for new team members.
- viii) Little or no use of software development tools or automation.
- ix) Highly dependent on individual skills and knowledge rather than standardized processes.
- x) High risk of project failure or delays due to a lack of process control and stability.

## Level 2: Repeatable

- i) Focuses on establishing basic project management policies.
- ii) Experience with earlier projects is used for managing new similar-natured projects.

### **iii) Project Planning:**

It includes defining resources required, goals, constraints, etc. for the project. It presents a detailed plan to be followed systematically for the successful completion of good-quality software.

### **iv) Configuration Management:**

The focus is on maintaining the performance of the software product, including all its components, for the entire lifecycle.

### **v) Requirements Management:**

It includes the management of customer reviews and feedback which result in some changes in the requirement set. It also consists of accommodation of those modified requirements.



**vi) Subcontract Management:**

It focuses on the effective management of qualified software contractors i.e. it manages the parts of the software developed by third parties.

**vii) Software Quality Assurance:**

It guarantees a good quality software product by following certain rules and quality standard guidelines while developing.

### **Level 3: Defined**

i) At this level, documentation of the standard guidelines and procedures takes place.

ii) It is a well-defined integrated set of project-specific software engineering and management processes.

**iii) Peer Reviews:**

In this method, defects are removed by using several review methods like walkthroughs, inspections, buddy checks, etc.

**iv) Intergroup Coordination:**

It consists of planned interactions between different development teams to ensure efficient and proper fulfillment of customer needs.

**v) Organization Process Definition:**

Its key focus is on the development and maintenance of standard development processes.

**vi) Organization Process Focus:**

It includes activities and practices that should be followed to improve the process capabilities of an organization.

**vii) Training Programs:**

It focuses on the enhancement of knowledge and skills of the team members including the developers and ensuring an increase in work efficiency.

## Level 4: Managed

- i) At this stage, quantitative quality goals are set for the organization for software products as well as software processes.
- ii) The measurements made help the organization to predict the product and process quality within some limits defined quantitatively.

### **iii) Software Quality Management:**

It includes the establishment of plans and strategies to develop quantitative analysis and understanding of the product's quality.

### **iv) Quantitative Management:**

It focuses on controlling the project performance quantitatively.

## Level 5: Optimizing

- i) This is the highest level of process maturity in CMM and focuses on continuous process improvement in the organization using quantitative feedback.
- ii) The use of new tools, techniques, and evaluation of software processes is done to prevent the recurrence of known defects.

### **iii) Process Change Management:**

Its focus is on the continuous improvement of the organization's software processes to improve productivity, quality, and cycle time for the software product.

### **iv) Technology Change Management:**

It consists of the identification and use of new technologies to improve product quality and decrease product development time.

### **v) Defect Prevention:**

It focuses on the identification of causes of defects and prevents them from recurring in future projects by improving project-defined processes.

## Q.7) Explain Software Quality Assurance. What is FTR?

**Ans:**

- i) Software Quality Assurance (SQA) is simply a way to assure quality in the software.
- ii) It is the set of activities that ensure processes, procedures as well as standards are suitable for the project and implemented correctly.
- iii) It is a process that works parallel to Software Development. It focuses on improving the process of development of software so that problems can be prevented before they become major issues.
- iv) Software Quality Assurance is a kind of Umbrella activity that is applied throughout the software process.
- v) For those looking to deepen their expertise in SQA and elevate their professional skills, consider exploring a specialized training program *Manual to Automation Testing: A QA Engineer's Guide*.
- vi) This program offers practical, hands-on experience and advanced knowledge that complements the concepts covered in this guide.

**FTR:**

- i) Formal Technical Review (FTR) is a software quality control activity performed by software engineers. It is an organized, methodical procedure for assessing and raising the standard of any technical paper, including software objects.
- ii) Finding flaws, making sure standards are followed, and improving the product or document under review's overall quality are the main objectives of a formal technical review (FTR).
- iii) Although FTRs are frequently utilized in software development, other technical fields can also employ the concept. In addition, the purpose of FTR is to enable junior engineers to observe the analysis, design, coding, and testing approach more closely.

- iv) FTR also works to promote backup and continuity to become familiar with parts of the software they might not have seen otherwise.
- v) FTR is a class of reviews that include walkthroughs, inspections, round-robin reviews, and other small-group technical assessments of software.
- vi) Each FTR is conducted as a meeting and is considered successful only if it is properly planned, controlled, and attended.

## Q.8) Explain FP Estimation techniques in details.

**Ans:**

Function Point (FP) estimation is one of the most widely used techniques for measuring the functional size of a software system. It was developed by Allan J. Albrecht at IBM in 1979 and later refined by the *International Function Point Users Group (IFPUG)* in 1984. The main objective of Function Point Estimation is to measure the software size based on the functionalities provided to the user rather than the lines of code written by the developer. It helps in estimating the total effort, cost, and time required for software development.

### **Objectives of Function Point Estimation:**

1. To provide a standardized method to measure software size.
2. To assist in project estimation in terms of cost, time, and resources.
3. To measure productivity and compare it with other projects.
4. To help in cost-benefit analysis and performance evaluation.
5. To provide a user-oriented measurement rather than a developer-oriented one.

### **Steps in Function Point Estimation Technique:**

#### **Step 1: Identify the Type of Functions**

The functions of a software system are divided into five types:

1. **External Inputs (EI):** These are data or control inputs entering the system from outside.  
Example: data entry screens.
2. **External Outputs (EO):** These are outputs generated by the system and sent outside.  
Example: reports or messages.
3. **External Inquiries (EQ):** These are input-output combinations used for data retrieval.  
Example: search or query functions.
4. **Internal Logical Files (ILF):** These are logical groups of data maintained within the system.  
Example: customer or employee database.

5. **External Interface Files (EIF):** These are files used by the system but maintained by other systems.  
Example: reference or master files from another application.

## Step 2: Assign Weights to Each Function Type

Each identified function is classified as Low, Average, or High complexity. Weights are assigned accordingly as shown below:

| Function Type                  | Low | Average | High |
|--------------------------------|-----|---------|------|
| External Inputs (EI)           | 3   | 4       | 6    |
| External Inquiries (EQ)        | 3   | 4       | 6    |
| External Outputs (EO)          | 4   | 5       | 7    |
| Internal Logical Files (ILF)   | 7   | 10      | 15   |
| External Interface Files (EIF) | 5   | 7       | 10   |

## Step 3: Calculate Unadjusted Function Points (UFP)

After assigning the weights, the Unadjusted Function Point (UFP) is calculated using the following formula:

$$\text{UFP} = \Sigma (\text{Number of each function type} \times \text{Weight assigned to it})$$

This gives the total unadjusted function points of the software system.

## Step 4: Determine the Value Adjustment Factor (VAF)

The Value Adjustment Factor (VAF) is determined based on 14 General System Characteristics (GSCs) such as:

1. Data communications
2. Performance

3. Heavily used configuration
4. Transaction rate
5. Online data entry
6. End-user efficiency
7. Complex processing
8. Reusability
9. Installation ease
10. Operational ease
11. Multiple sites
12. Changeability
13. Distributed functions
14. System complexity

Each characteristic is rated on a scale from 0 to 5, where 0 means no influence and 5 means strong influence. The Value Adjustment Factor is calculated using the formula:

$$\text{VAF} = 0.65 + (0.01 \times \text{Total Degree of Influence})$$

### **Step 5: Calculate Adjusted Function Points (AFP)**

Once the UFP and VAF are known, the Adjusted Function Point (AFP) is calculated using the following formula:

$$\text{AFP} = \text{UFP} \times \text{VAF}$$

This gives the final Function Point value that represents the overall functional size of the software.

### **Step 6: Effort Estimation Using Function Points**

After calculating AFP, the total effort required for software development can be estimated as follows:

$$\text{Effort (Person-Hours)} = \text{AFP} \times \text{Productivity Rate}$$

Where *Productivity Rate* is the average number of function points developed per person-month, which varies from organization to organization.

## **Q.9) Explain Agile process model of software development.**

**Ans:**

- i) The Agile Process Model is an iterative and incremental approach to software development that emphasizes flexibility, collaboration, and customer satisfaction.
- ii) It was primarily designed to help projects adapt quickly to changing requirements and deliver working software in short cycles.
- iii) The main aim of the Agile model is quick project completion with continuous feedback and improvement.
- iv) Agility in this model is achieved by fitting the process to the project and removing unnecessary activities that do not add value. Anything that wastes time or effort is avoided.
- v) The Agile model is not a single process but a group of related methodologies such as *Scrum*, *Extreme Programming (XP)*, and *Crystal Models*, which share common principles.

### **Phases in Agile Model:**

#### **1. Requirement Gathering:**

- In this phase, requirements are gathered through continuous interaction with the customer.
- The team identifies key features, business goals, and feasibility (technical and economic) of the project.
- The focus is on understanding user needs rather than preparing lengthy documents.

#### **2. Design the Requirements:**

- Based on the gathered requirements, the system design is prepared using wireframes and high-level UML diagrams.
- The design shows how features will function and integrate with existing systems.
- Prototypes are created to get early user feedback before development begins.

#### **3. Construction / Iteration:**

- The software is developed in small cycles known as iterations.
- Each iteration lasts 1–4 weeks and results in a working version of the product.



- New features are coded, tested, and integrated during each iteration, gradually improving the product.
- 4. **Testing / Quality Assurance:**
  - Testing is performed continuously throughout the development process. It includes:
    - *Unit Testing*: Testing individual units or modules.
    - *Integration Testing*: Ensuring that combined modules work together.
    - *System Testing*: Verifying that the overall software meets user requirements and performs as expected.
- 5. **Deployment:**
  - After testing, the working software is deployed to the end users.
  - In Agile, deployment is *incremental*, meaning updates and improvements are delivered regularly after each iteration.
  - Feedback from users is immediately incorporated in the next cycle.
- 6. **Feedback:**
  - Feedback from customers and stakeholders is collected after each release.
  - This feedback helps the development team correct errors, improve features, and plan the next iteration more effectively.
  - Continuous improvement based on feedback is a key aspect of Agile.

### **Characteristics of Agile Model:**

1. Highly adaptable to changing requirements and technologies.
2. Incremental development, each iteration delivers a usable product.
3. Continuous customer involvement and feedback.
4. Focus on working software rather than extensive documentation.
5. Frequent delivery of product increments in short time periods.
6. Emphasizes teamwork, communication, and collaboration among developers and clients.
7. Encourages regular reflection and process improvement after each iteration.

## Q.10) Explain the Spiral model of Software development.

Ans:

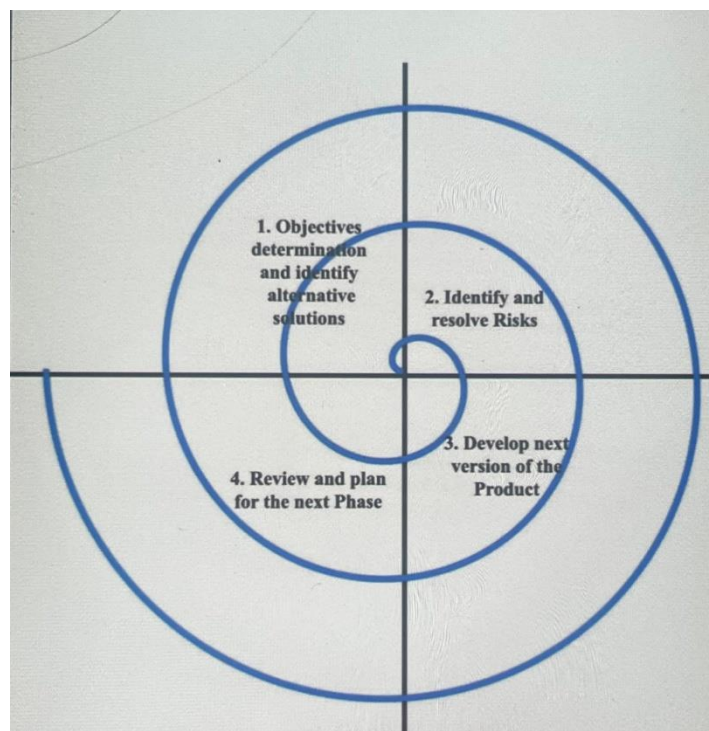
The Spiral Model is a *Software Development Life Cycle (SDLC)* model that provides a systematic and iterative approach to software development. In its diagrammatic representation, it looks like a spiral with many loops. The exact number of loops of the spiral is unknown and can vary from project to project. Each loop of the spiral is called a *phase* of the software development process.

### Some Key Points regarding the Stages of a Spiral Model:

1. The exact number of phases needed to develop the product can be varied by the project manager depending upon the project risks.
2. As the project manager dynamically determines the number of phases, the project manager has an important role in developing a product using the spiral model.
3. It is based on the idea of a spiral, with each iteration of the spiral representing a complete software development cycle, from requirements gathering and analysis to design, implementation, testing, and maintenance.

### Phases of the Spiral Model:

The Spiral Model is a *risk-driven model*, meaning that the focus is on managing risk through multiple iterations of the software development process. **Each phase**



**of the Spiral Model is divided into four Quadrants:**

### **1. Objectives Defined**

- i) In the first phase of the spiral model, we clarify what the project aims to achieve, including functional and non-functional requirements.
- ii) Requirements are gathered from the customers and the objectives are identified, elaborated, and analysed at the start of every phase. Then alternative solutions possible for the phase are proposed in this quadrant.

### **2. Risk Analysis and Resolving**

- i) In the risk analysis phase, the risks associated with the project are identified and evaluated.
- ii) During the second quadrant, all the possible solutions are evaluated to select the best possible solution. Then the risks associated with that solution are identified and the risks are resolved using the best possible strategy. At the end of this quadrant, the Prototype is built for the best possible solution.

### **3. Develop the next version of the Product**

- i) During the third quadrant, the identified features are developed and verified through testing.  
At the end of the third quadrant, the next version of the software is available.
- ii) In the evaluation phase, the software is evaluated to determine if it meets the customer's requirements and if it is of high quality.

### **4. Review and plan for the next Phase**

- i) In the fourth quadrant, the Customers evaluate the so-far developed version of the software.  
In the end, planning for the next phase is started.
- ii) The next iteration of the spiral begins with a new planning phase, based on the results of the evaluation.

iii) The Spiral Model is often used for complex and large software development projects, as it allows for a more flexible and adaptable approach to *Software development*. It is also well-suited to projects with significant uncertainty or high levels of risk.

iv) The Radius of the spiral at any point represents the expenses (cost) of the project so far, and the angular dimension represents the progress made so far in the current phase.

### **Q.11) Explain SCRUM Methodology with suitable diagram.**

**Ans:**

i) Scrum is an Agile methodology used for managing and controlling software development projects.

ii) It is a lightweight, iterative, and incremental framework that enables teams to work together, self-organize, and adapt to changing requirements quickly.

iii) The main goal of the Scrum methodology is to deliver high-value products in short time frames while maintaining creativity, quality, and productivity.

iv) Scrum is a management and development framework that helps teams address complex adaptive problems while efficiently delivering products of the highest possible value.

### **Key Features of Scrum:**

1. Scrum is a lightweight and simple framework.
2. It emphasizes self-organization and teamwork.
3. It follows an iterative and incremental approach.
4. It promotes continuous feedback and improvement.
5. It ensures fast, flexible, and customer-focused delivery.

### **Phases / Lifecycle of Scrum Methodology:**

The Scrum methodology follows a cyclic process known as the Scrum Lifecycle, which includes the following stages:

## 1. Product Backlog:

- It is a prioritized list of all desired features, enhancements, and fixes required in the product.
- The Product Owner maintains the backlog and keeps it updated according to user requirements and feedback.

## 2. Sprint Planning:

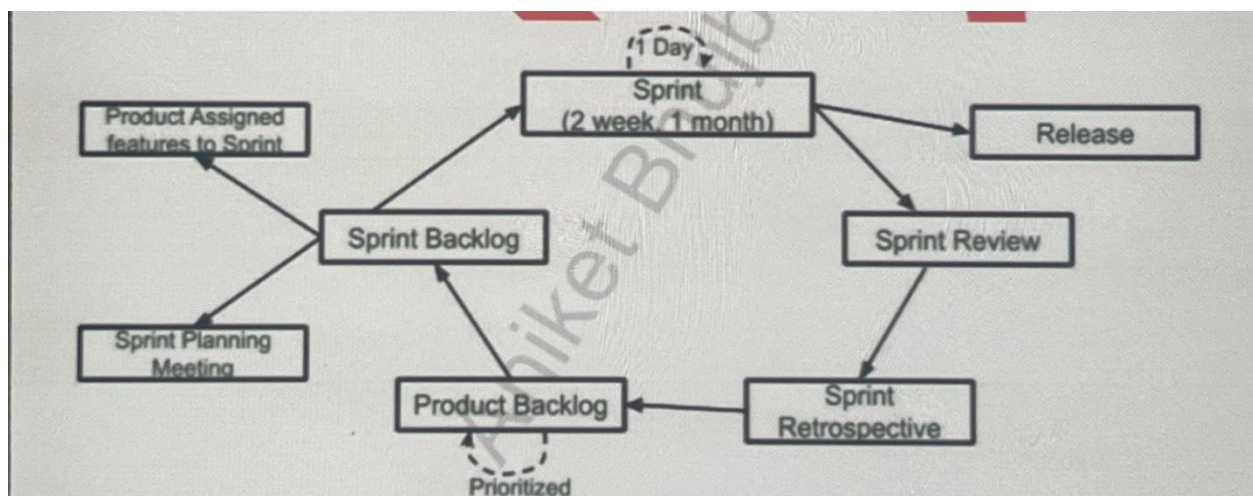
- A meeting is conducted at the beginning of each sprint to select tasks from the Product Backlog that will be completed in that sprint.
- These selected items form the **Sprint Backlog**.

## 3. Sprint (Iteration):

- A Sprint is a time-boxed development cycle, usually lasting **1 to 4 weeks**.
- During a sprint, the team develops, tests, and integrates features into a working version of the product.
- At the end of each sprint, a usable product increment is produced.

## 4. Daily Scrum (Daily Stand-Up Meeting):

- A short **15-minute meeting** conducted daily to discuss progress, challenges, and upcoming tasks.
- It improves communication and keeps the team synchronized.



## **5. Sprint Review:**

- Conducted at the end of each sprint to demonstrate the completed work to the stakeholders.
- Feedback from this meeting is recorded and used for future improvements.

## **6. Sprint Retrospective:**

- After the Sprint Review, the team reflects on what went well and what needs improvement.
- This helps in continuously refining the process and increasing team efficiency.

## **7. Release:**

- Once the required features are complete and approved, the product is released to the customer or market.
- Deployment ensures that the product is available for end-users.

## Q.12) Explain Software Reverse Engineering in detail.

**Ans:** i) Software Reverse Engineering is a process of recovering the design, requirement specifications, and functions of a product from an analysis of its code.

ii) It builds a program database and generates information from this.

iii) Reverse engineering can extract design information from source code, but the abstraction level, the completeness of the documentation, the degree to which tools and a human analyst work together, and the directionality of the process are highly variable.

### Objective of Reverse Engineering:

1. **Reducing Costs:** Reverse engineering can help cut costs in product development by finding replacements or cost-effective alternatives for systems or components.
2. **Analysis of Security:** Reverse engineering is used in cybersecurity to examine exploits, vulnerabilities, and malware. This helps in understanding the threat mechanisms and the development of practical defences by security experts.
3. **Integration and Customization:** Through the process of reverse engineering, developers can incorporate or modify hardware or software components into pre-existing systems to improve their operation or tailor them to meet particular needs.
4. **Recovering Lost Source Code:** Reverse engineering can be used to recover the source code of a software application that has been lost or is inaccessible or at the very least, to produce a higher-level representation of it.
5. **Fixing bugs and maintenance:** Reverse engineering can help find and repair flaws or provide updates for systems for which the original source code is either unavailable or inadequately documented.

### Reverse Engineering to Understand Processing:

To understand processing begins with an attempt to understand and then extract procedural abstractions represented in the source code. To understand procedural abstractions, the code is analysed at varying levels of abstraction: system, program, component, pattern, and statement.

1. Each of the programs that make up the application system represents a functional abstraction at a high level of detail. A block diagram, representing the interaction between these functional abstractions, is created.
2. Each component performs some sub function and represents a defined procedural abstraction. A processing narrative for each component is developed.

For large systems, reverse engineering is generally accomplished using a semi automated (partial automation) approach. Automated tools can be used to help you understand the semantics of existing code. The output of this process is then passed to restructuring and forward engineering tools to complete the reengineering process.

## **Steps of Software Reverse Engineering:**

1. **Collection Information:**  
This step focuses on collecting all possible information (i.e., source design documents, etc.) about the software.
2. **Examining the Information:**  
The information collected in step-1 is studied so as to get familiar with the system.
3. **Extracting the Structure:**  
This step concerns identifying program structure in the form of a structure chart where each node corresponds to some routine.
4. **Recording the Functionality:**  
During this step, processing details of each module of the structure, charts are recorded using structured language like decision table, etc.
5. **Recording Data Flow:**  
From the information extracted in step-3 and step-4, a set of data flow diagrams is derived to show the flow of data among the processes.
6. **Recording Control Flow:**  
The high-level control structure of the software is recorded.
7. **Review Extracted Design:**  
The design document extracted is reviewed several times to ensure consistency and correctness. It also ensures that the design represents the program.
8. **Generate Documentation:**  
Finally, in this step, the complete documentation including SRS, design document, history, overview, etc. is recorded for future use.



### Q.13) What is cost Estimation? Explain LOC model.

**Ans:**

- i) **Cost estimation** simply means a technique that is used to find out the cost estimates.
- ii) The cost estimate is the financial spend that is done on the efforts to develop and test software in Software Engineering.
- iii) Cost estimation models are some mathematical algorithms or parametric equations that are used to estimate the cost of a product or a project.
- iv) Various techniques or models for cost estimation, also known as Cost Estimation Models.

#### **LOC (Lines of Code) Model:**

- i) The **Lines of Code (LOC) Model** is one of the earliest and simplest software cost estimation techniques.
- ii) It is a **size-oriented metric** that estimates the effort and cost of a software project based on the total number of lines of source code to be written.
- iii) A **Line of Code (LOC)** is any line in the source code that is not a comment or blank line.
- iv) It includes header files, variable declarations, and all executable and non-executable statements.
- v) In short, every meaningful line that contributes to the program logic is counted as LOC.

#### **Features of Lines of Code (LOC):**

##### **1. Change Tracking:**

LOC can be used to track the growth or reduction of a codebase over time. Variations in LOC help in analysing project progress and estimating development effort.

2. **Ease of Computation:**

LOC is a simple and quick measure to calculate, as it only involves counting the non-comment lines in the program.

3. **Ease of Understanding:**

LOC is an intuitive measure that can be easily understood by both technical and non-technical stakeholders. Expressing code size in terms of lines provides a clear and simple representation.

4. **Limited Representation of Complexity:**

LOC only measures the volume of code, not the complexity. Two programs may have the same number of lines but differ greatly in complexity or functionality.

5. **Comparability:**

LOC is best used for comparing or estimating projects that use the **same programming language and coding standards**, since different languages may vary in verbosity.

6. **Benchmarking Tool:**

LOC can act as a benchmark to compare various versions or releases of the same software and study how modifications affect overall code size and maintainability.

7. **Effort Estimation Correlation:**

Research shows a rough correlation between LOC and the effort, cost, duration, and number of defects in a software project. A lower LOC value generally indicates better efficiency and maintainability.

## Steps in LOC-Based Cost Estimation:

1. **Estimate the total number of lines of code** expected in the software (LOC or KLOC, where KLOC = 1000 lines of code).
2. **Determine the productivity rate**, i.e., the number of lines of code produced per person-month.
3. **Compute the effort** using the empirical formula:

$$\text{Effort (Person-Months)} = a + b \times (\text{KLOC})$$

where **a** and **b** are constants based on past project data.

4. **Estimate development time and total cost** as:

$$\text{Development Time (Months)} = c \times (\text{Effort})^d$$

$$\text{Cost} = \text{Effort} \times \text{Cost per Person-Month}$$

5. **Validate** the estimation considering team skill, complexity, and tools used.

### **Example:**

If a project has **20 KLOC**, and constants are:

**a = 2.4, b = 1.05, c = 2.5, d = 0.38**

Then,

**Effort =  $2.4 + 1.05 \times 20 = 23.4$  Person-Months**

If cost per person-month = ₹50,000, then

**Total Cost =  $23.4 \times 50,000 = ₹11,70,000$**

## Q.14) Explain the version control and change control along with its steps.

**Ans:**

### **Version Control:**

Version control (also known as revision control or source code control) is a set of procedures and tools used within **Software Configuration Management (SCM)** for managing the creation, evolution, and use of multiple versions of configuration objects (such as code files, documentation, libraries) stored in an SCM repository.

A version control system enables the team to keep a history of all changes, support parallel work, revert to previous versions, and build specific versions of the product as needed.

### **Key capabilities of version control include:**

- An SCM repository that stores all relevant configuration objects.
- A version management facility that stores all revisions (versions) of each configuration object.
- A “make” capability or build facility that allows the software engineer to collect relevant configuration objects (versions) and construct a specific version of the software.
- Issue or bug-tracking capability that allows the team to record and manage outstanding issues associated with each configuration object or version.

The SCM repository also maintains a **changeset** — a collection of all the changes made to a baseline configuration.

The changeset captures who made each change, when, and why, and is used to build a specific version of the software.

## Steps / Typical Workflow for Version Control:

While version control is tool-driven, you can list a typical workflow:

1. **Check-out / Update:**  
A developer obtains the latest version of the configuration object from the repository into their local workspace.
2. **Modify:**  
Developer makes changes (code edits, new documentation, bug fixes) in the workspace.
3. **Commit / Check-in:**  
The developer records the changes back into the repository, creating a new version/revision of the object, including metadata (who, when, reason).
4. **Build / Merge:**  
The system or team integrates multiple changes (branches) into a combined version, resolves conflicts if needed, and uses the make/build facility to construct a coherent version.
5. **Tag / Label / Baseline:**  
The version is given a label or tag, designating it as a baseline, release, or checkpoint.
6. **Revert / Rollback:**  
If any problem is found, the team may revert to a previous stable version from the repository history.
7. **Audit / Trace:**  
The repository allows tracing of versions, authors, dates, change sets, and supports configuration audits.
- 8.

## Change Control (Change Management):

Change control is the process within configuration management that identifies, evaluates, approves, implements, and monitors changes to configuration items, baselines, or project plans.

The objectives of change control are to:

- Maintain the integrity of the product configuration.
- Ensure that changes are reviewed and approved before implementation.
- Ensure that the documentation and plan remain consistent.

## Steps in Change Control Process:

Here is a standard sequence of steps for change control:

1. **Change Request / Identification:**

A change need is recognized (new requirement, defect, external driver) and a formal **Request for Change (RFC)** is submitted, documenting what needs to change and why.

2. **Review & Assessment:**

The change control board or designated authority reviews the RFC, checks feasibility, impacts (cost, schedule, quality, risk), and prioritizes the change.

3. **Planning the Change:**

A detailed implementation plan is prepared: what will be done, by whom, when, dependencies, rollback plan.

4. **Approval / Authorization:**

The change is formally approved (or rejected) by the authority (Change Control Board) based on the evaluation.

5. **Implementation / Execution:**

The change is executed: code modified, tests run, documentation updated, configuration objects updated in version control.

6. **Verification / Testing:**

The implemented change is validated and verified to meet the requirements and ensure no adverse side-effects.

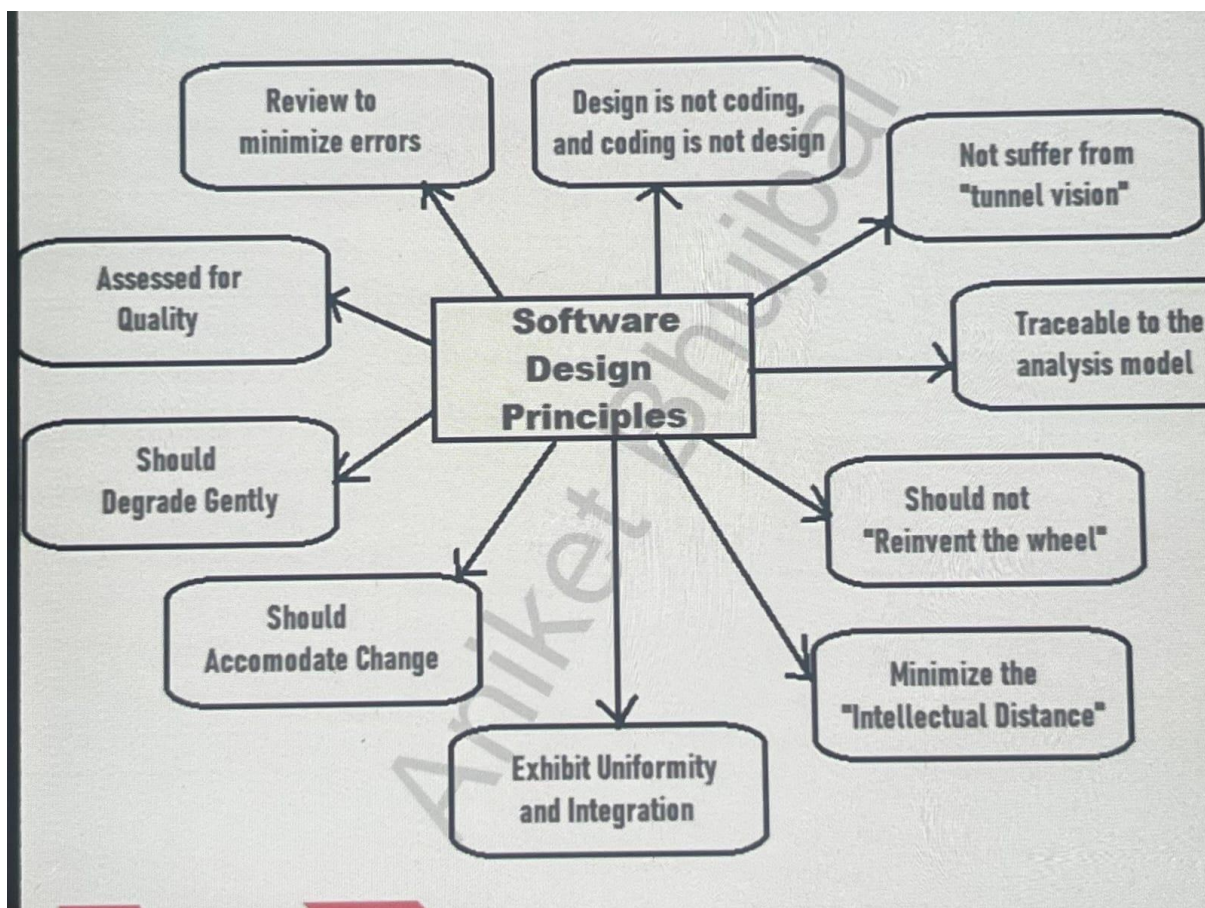
7. **Closure:**

After successful verification, the change request is closed, and records updated (change logs, impact data, lessons learned).  
Plan and baselines are updated as necessary.

### Q.15) List and explain the principle of software design with example.

Ans:

- i) **Software Design** is also a process to plan or convert the software requirements into a step that are needed to be carried out to develop a software system.
- ii) There are several principles that are used to organize and arrange the structural components of Software design.
- iii) Software Designs in which these principles are applied affect the content and the working process of the software from the beginning.
- iv) These principles are stated below:



- 1. Should not suffer from “Tunnel Vision”** – While designing the process, it should not suffer from “tunnel vision” which means that it should not only focus on completing or achieving the aim but on other effects also.
- 2. Traceable to analysis model** – The design process should be traceable to the analysis model which means it should satisfy all the requirements that software requires to develop a high-quality product.
- 3. Should not “Reinvent the Wheel”** – The design process should not reinvent the wheel that means it should not waste time or effort in creating things that already exist. Due to this, the overall development will get increased.
- 4. Minimize intellectual distance** – The design process should reduce the gap between real-world problems and software solutions for that problem meaning it should simply minimize intellectual distance.
- 5. Exhibit uniformity and integration** – The design should display uniformity which means it should be uniform throughout the process without any change. Integration means it should mix or combine all parts of software i.e. subsystems into one system.
- 6. Accommodate change** – The software should be designed in such a way that it accommodates the change implying that the software should adjust to the change that is required to be done as per the user’s need.
- 7. Degrade gently** – The software should be designed in such a way that it degrades gracefully which means it should work properly even if an error occurs during the execution.
- 8. Assessed or quality** – The design should be assessed or evaluated for the quality meaning that during the evaluation, the quality of the design needs to be checked and focused on.
- 9. Review to discover errors** – The design should be reviewed which means that the overall evaluation should be done to check if there is any error present or if it can be minimized.
- 10. Design is not coding and coding is not design** – Design means describing the logic of the program to solve any problem and coding is a type of language that is used for the implementation of a design.



## **Q.16) Explain the tracking and scheduling & what are the different types of software maintenance?**

**Ans:**

### **1. Scheduling:**

Scheduling is the process of planning when and how maintenance tasks will be performed. It involves identifying all maintenance activities, estimating the time and effort required for each task, and deciding the sequence of operations. Proper scheduling ensures that all maintenance activities such as debugging, testing, documentation updates, and enhancements are completed within a defined time frame.

#### **Key points about scheduling:**

- It defines the order in which maintenance activities should occur.
- Resources such as developers, testers, and tools are allocated properly.
- It helps in setting deadlines and milestones for each phase of maintenance.
- Prevents delays and ensures smooth workflow and timely delivery of updates.
- Helps in balancing workload and avoiding resource conflicts.

An efficient maintenance schedule ensures that critical updates, bug fixes, and security patches are implemented at the right time without affecting the system's performance or user operations.

### **2. Tracking:**

Tracking is the process of monitoring and controlling maintenance activities as per the schedule. It involves comparing the planned progress with the actual progress to identify any deviations or delays. Through tracking, managers can take corrective actions if tasks are behind schedule or if there are quality issues in maintenance work.

#### **Key points about tracking:**

- Tracks actual progress against planned targets and milestones.
- Detects delays, cost overruns, or quality issues in time.

- Helps maintain transparency and accountability among team members.
- Provides early warning signals for potential risks or failures.
- Supports decision-making for reallocation of resources or schedule adjustments.

Together, **tracking and scheduling** ensure that maintenance activities are systematically managed, well-organized, and completed on time, maintaining software stability and user satisfaction.

## Types of Software Maintenance:

Software maintenance means the process of modifying a software product after it has been delivered to the customer.

There are mainly **four types** of software maintenance:

### 1. Corrective Maintenance:

- It involves fixing the errors or defects that are found after the software has been delivered.
- Example: If a calculation in the software gives the wrong result, the correction of that error is part of corrective maintenance.

### 2. Adaptive Maintenance:

- It is done to make the software adaptable to a new environment.
- Example: When the operating system or hardware is changed, software may need modification to work with the new system.

### 3. Perfective Maintenance:

- It involves improving the performance or enhancing the functionalities of the software according to the user's new requirements.
- Example: Adding a new feature in the existing software.

### 4. Preventive Maintenance:

- It is done to prevent future problems in the software by improving its reliability and maintainability.
- Example: Restructuring the code to make it more efficient and less error-prone.

## 5. Patching:

- **Patching** is a type of corrective maintenance activity.
- It refers to the process of making small changes or updates to the software to fix specific problems, such as bugs or security vulnerabilities, without changing the overall structure of the system.
- Patches are usually released by software developers after deployment to correct issues reported by users.
- Example: Updating an antivirus software to fix a newly discovered malware threat.

## Q.17) How user Interface Design helps web technology.

**Ans:**

- i) User Interface (UI) Design plays a vital role in the success of web technology as it determines how effectively a user can interact with a website or web application.
- ii) A well-designed UI enhances the user experience, increases user satisfaction, and ensures smooth communication between the user and the system.
- iii) In web technology, the UI acts as the bridge between complex backend processes and the end user, making technology accessible and usable to everyone.
- iv) A good user interface should be attractive, simple to use, responsive, consistent, and easy to understand. When users find a website visually appealing and easy to navigate, they are more likely to stay longer, explore more, and revisit the site.

**The importance and role of UI design in web technology can be understood as follows:**

### **1. Improves User Experience (UX):**

The primary purpose of UI design is to make the interaction between users and the website smooth and enjoyable.

A good UI provides clear layouts, meaningful visuals, and well-organized content that make it easier for users to find information and complete their tasks efficiently.

### **2. Enhances Accessibility:**

A properly designed interface ensures that websites are accessible to all kinds of users, including those with disabilities or limited technical knowledge.

Accessibility features such as readable fonts, color contrast, and screen reader compatibility help make the web more inclusive.

### **3. Increases Efficiency and Usability:**

A simple and well-organized interface helps users perform actions faster without confusion.

When menus, buttons, and forms are designed properly, users can navigate easily, saving time and reducing frustration.

### **4. Builds Brand Identity and Trust:**

A visually appealing and consistent UI design reflects professionalism.

It helps in building the brand image of a website or organization.

When users find a site aesthetically pleasing and easy to use, they tend to trust it more.

### **5. Supports Multi-Platform Compatibility:**

In web technology, users access websites from different devices such as desktops, laptops, tablets, and smartphones.

A responsive UI ensures that the website adjusts itself automatically to different screen sizes, providing the same level of comfort and experience across all platforms.

### **6. Reduces Errors and Confusion:**

A clear interface design helps users understand what actions they can take and what results to expect.

Proper labels, instructions, and feedback messages reduce user mistakes and improve task completion rates.

### **7. Boosts User Engagement:**

Interactive and visually appealing designs attract users' attention.

When users find a website interesting and easy to explore, they spend more time on it, which increases engagement and conversion rates.

### **8. Simplifies Complex Functions:**

Through intuitive design, even complex web functions can be made easy to use.

For example, drag-and-drop features, icons, and visual cues help users perform tasks without needing technical knowledge.

### **9. Improves Business and Productivity:**

In web-based applications, a good UI saves time for both users and businesses by improving efficiency and reducing support needs.

Satisfied users are more likely to become loyal customers, which benefits the business in the long run.

### **10. Follows Design Principles and Process:**

The UI design process includes stages like **user analysis, interface design, implementation, and validation.**

- In **User Analysis**, designers study the target audience and their needs.
- In **Interface Design**, layouts, colors, and controls are decided.
- **Implementation** involves developing and testing prototypes.
- Finally, **Validation** ensures that the interface meets user expectations and performs efficiently.

## **Q.18) Explain the XP. Explain the development of use case.**

**Ans:**

i) **Extreme Programming (XP)** is an Agile software development methodology that focuses on delivering high-quality software through frequent and continuous feedback, collaboration, and adaptation.

ii) XP emphasizes a close working relationship between the development team, the customer, and stakeholders, with an emphasis on rapid, iterative development and deployment.

iii) Agile development approaches evolved in the 1990s as a reaction to documentation and bureaucracy-based processes, particularly the waterfall approach.

**iv) Agile approaches are based on some common principles, some of which are:**

1. Working software is the key measure of progress in a project.
2. For progress in a project, therefore software should be developed and delivered rapidly in small increments.
3. Even late changes in the requirements should be entertained.
4. Face-to-face communication is preferred over documentation.
5. Continuous feedback and involvement of customers are necessary for developing good agile methods.
6. A simple design that involves and improves with time is a better approach than doing an elaborate design upfront for handling all possible scenarios.
7. The delivery dates are decided by empowered teams of talented individuals.
8. Extreme programming is one of the most popular and well-known approaches in the family of agile methods.
9. An XP project starts with user stories which are short descriptions of what scenarios the customers and users would like the system to support.
10. Each story is written on a separate card, so they can be flexibly grouped.

## Use Case:

A **use case** is a technique used in software and systems engineering to capture the functional requirements of a system.

It describes how a user (known as an actor) interacts with the system to achieve a specific goal.

The development of use cases helps in understanding system behavior from a user's perspective and ensures that all possible interactions are considered during design.

## The steps involved in the development of a use case are as follows:

### 1. Identify Actors:

The first step is to identify all the users or external systems that will interact with the system.

Each actor represents a role that interacts with the system, such as a customer, administrator, or third-party service.

### 2. Identify Use Cases:

After identifying the actors, determine what each actor wants to achieve with the system.

These goals or interactions become the use cases (for example, *login*, *register*, *place order*, *generate report*, etc.).

### 3. Establish Relationships:

Define how actors and use cases are connected.

This step involves showing which actor performs which use case.

Relationships can also exist between use cases using *include*, *extend*, or *generalization* relationships.

### 4. Describe the Use Case:

Write a detailed description for each use case.

This includes:

- **Use case name**
- **Actors involved**
- **Preconditions** (what must be true before the use case starts)

- **Main flow** (step-by-step description of the normal scenario)
- **Alternate flow** (variations or exceptions)
- **Post conditions** (the result after execution)

### 5. Draw the Use Case Diagram:

Represent all use cases and actors visually using ovals for use cases and stick figures for actors.

Connect actors and use cases with lines to show their interactions.

This diagram helps in visualizing the overall functionality of the system at a glance.

### 6. Review and Validate:

Finally, review the use cases with stakeholders to ensure they correctly represent the required system behaviour.

Any missing scenarios or interactions are added, and unnecessary ones are removed.

## Q.19) Explain System Process Framework.

**Ans:**

i) A **Software Process Framework** is a structured approach that defines the steps, tasks, and activities involved in software development.

- It divides the entire development process into organized stages to ensure clarity and order.
- This structure helps developers and managers understand what to do, when to do it, and how to do it efficiently.

ii) This framework serves as a **foundation for software engineering**, guiding the development team through various stages to ensure a systematic and efficient process.

- It helps teams follow a consistent development pattern from requirements to maintenance.
- Every phase is interconnected, ensuring no important activity is missed during development.



iii) A **Software Process Framework** helps in **project planning, risk management, and quality assurance** by detailing the chronological order of actions.

- It assists in identifying possible risks before they occur and preparing strategies to handle them.
- By defining steps and sequence, it ensures that the project is completed within time and meets quality standards.

iv) It includes **task sets, umbrella activities, and process framework activities**, all essential for a successful software development lifecycle.

- **Task sets** define the group of related tasks that fulfill specific goals.
- **Umbrella activities** like documentation, quality assurance, and project tracking continue throughout the project.
- **Process framework activities** are high level phases like communication, planning, modelling, construct and deployment.

v) **Software Process Framework** enhances **productivity, consistency, and quality** of the overall software development process.

- A consistent process minimizes confusion, reduces errors, and helps deliver the product within budget and schedule.
- It details the steps and chronological order of a process.

vi) It acts as the foundation for most software development methodologies like **Waterfall, Spiral, and Agile**.

- It ensures that each step is performed logically, improving project monitoring and control.

vii) **Task sets, umbrella activities, and process framework activities** define the characteristics of the software development process.

- They ensure that every part of the development process—from requirements to deployment—is properly managed.

viii) **Tasks** focus on small, specific objectives.

- Each task represents a single piece of work such as writing code, testing a function, or collecting data.

- These tasks combine to create the final deliverables of the project.

ix) **Action** is a set of related tasks that produce a **major work product**.

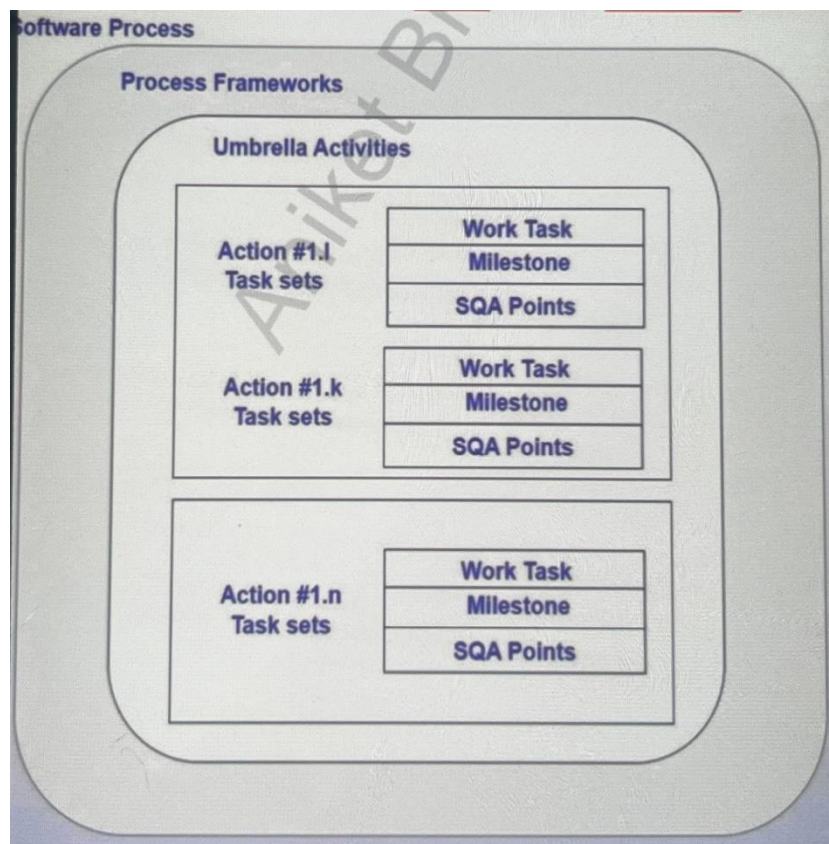
- Example: Designing a database involves multiple tasks like creating tables, setting relationships, and defining keys.
- Together, these tasks form one major output called the “database design.”

x) **Activities** are groups of related actions that achieve a major project objective.

- Activities represent large phases like “Requirement Analysis,” “Design,” “Implementation,” or “Testing.”
- Each activity includes several smaller actions and tasks that lead to the successful completion of that phase.

xi) A **Software Process Framework** ensures the **systematic flow of development**, from requirement gathering to maintenance.

- It provides structure to large projects, making management and monitoring easier.
- It improves coordination among teams and enhances the final software quality.



## Q.20) Explain Requirement Model.

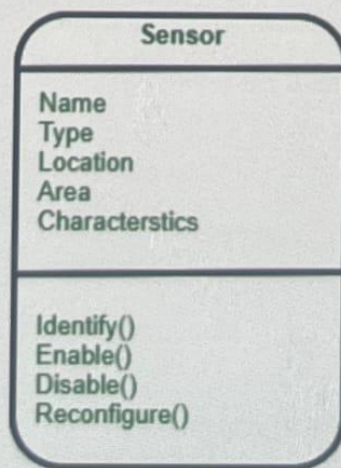
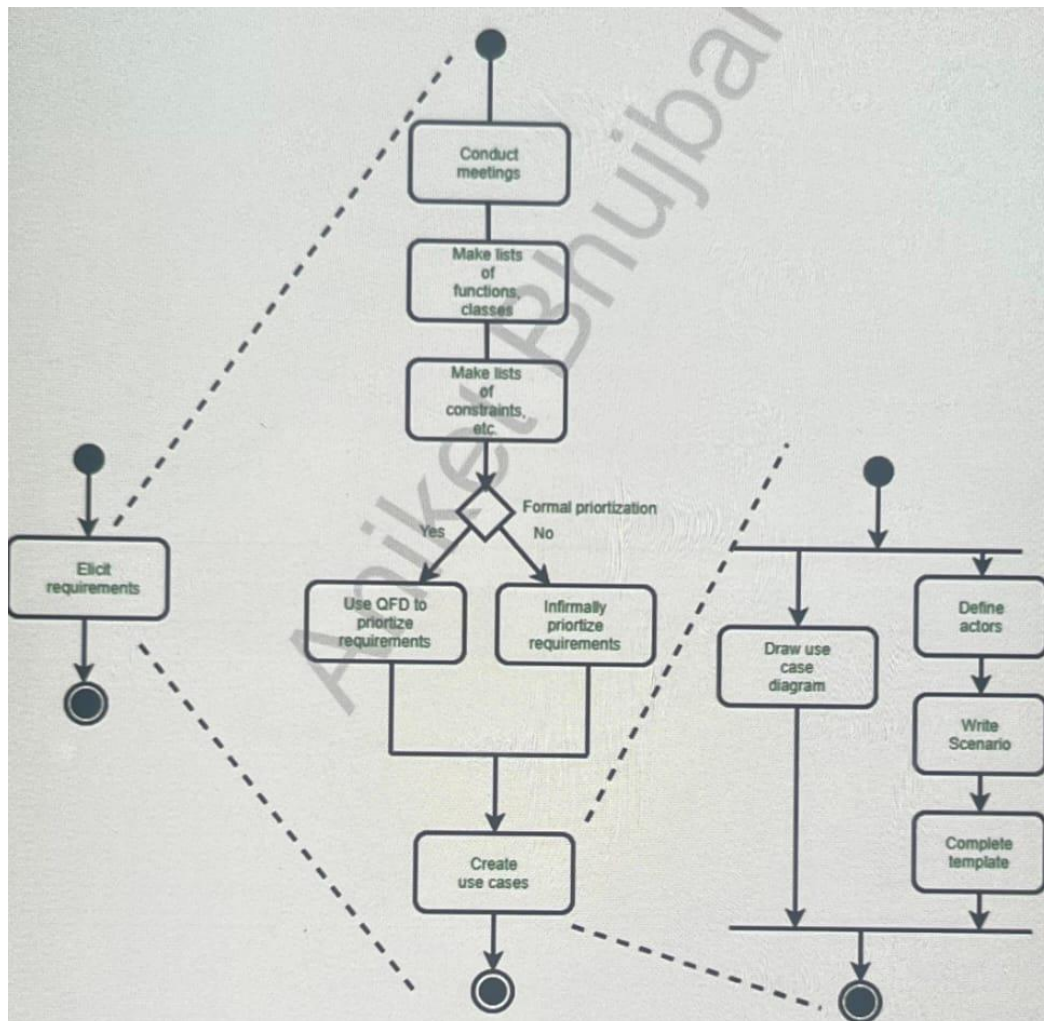
Ans:

**Requirements for a computer-based system** can be seen in many different ways. Some software people argue that it's worth using a number of different modes of representation while others believe that it's best to select one mode of representation. The specific

### **elements of the requirements model**

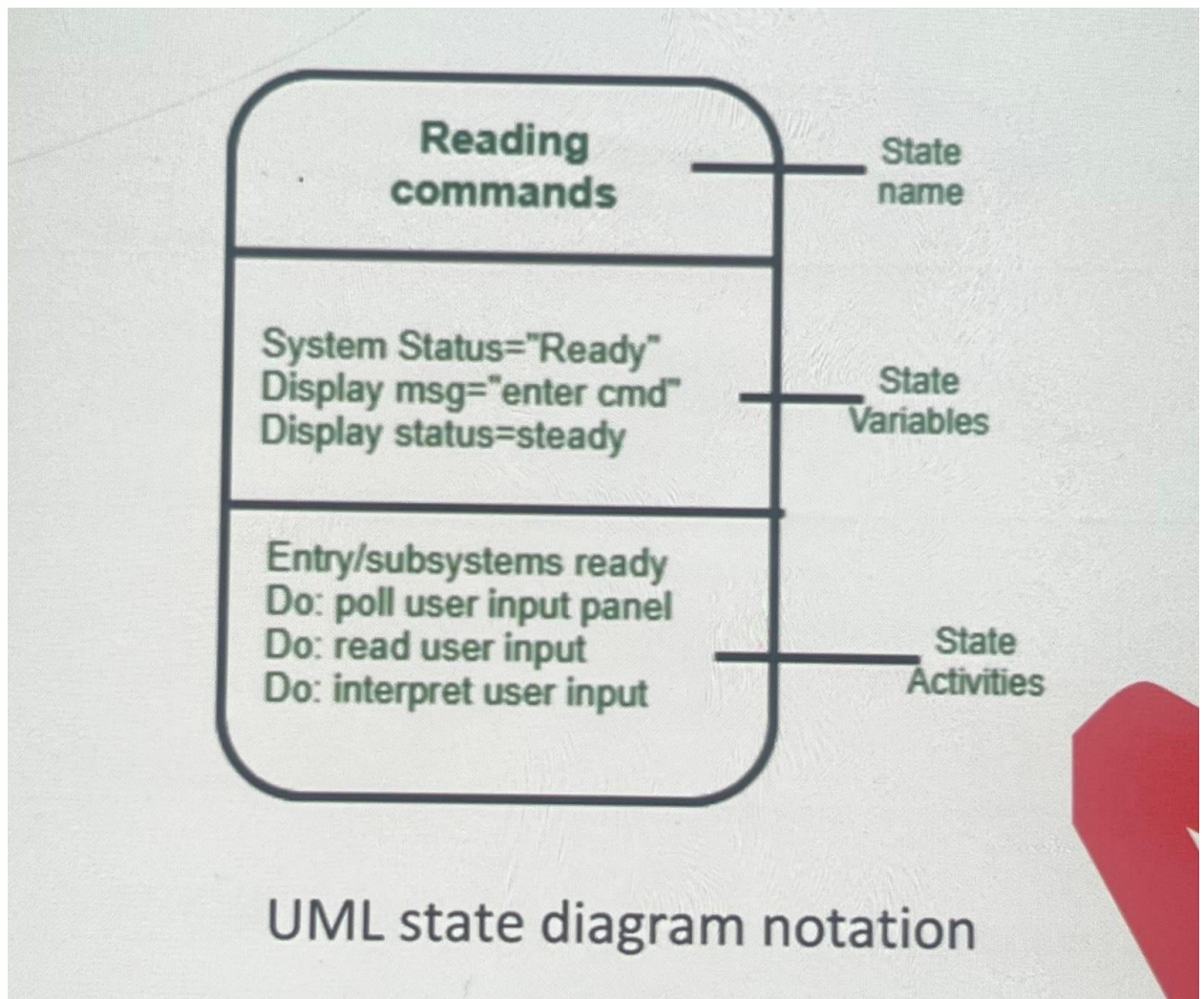
are dedicated to the analysis modelling method that is to be used.

- **Scenario-based elements:** Using a scenario-based approach, system is described from user's point of view. *For example*, basic use cases and their corresponding use-case diagrams evolve into more elaborate template-based use cases. Figure 1(a) depicts a UML activity diagram for eliciting requirements and representing them using use cases. There are three levels of elaboration.
- **Class-based elements:** A collection of things that have similar attributes and common behaviours i.e., objects are categorized into classes. *For example*, a UML case diagram can be used to depict a Sensor class for the Safe Home security function. Note that diagram lists attributes of sensors and operations that can be applied to modify these attributes. In addition to class diagrams, other analysis modelling elements depict manner in which classes collaborate with one another and relationships and interactions between classes.
- **Behavioural elements:** Effect of behaviour of computer-based system can be seen on design that is chosen and implementation approach that is applied. Modelling elements that depict behaviour must be provided by requirements model.



Class diagram for sensor





Method for representing behaviour of a system by depicting its states and events that cause system to change state is state diagram. A state is an externally observable mode of behaviour. In addition, state diagram indicates actions taken as a consequence of a particular event. To illustrate use of a state diagram, consider software embedded within Safe Home control panel that is responsible for reading user input. A simplified UML state diagram is shown in figure 2.

- **Flow-oriented elements:** As it flows through a computer-based system information is transformed. System accepts input, applies functions to transform it, and produces output in various forms. Input may be a control signal transmitted by a transducer, a series of numbers typed by human operator, a packet of information transmitted on a network link, or a voluminous data file retrieved from secondary storage. Transform may comprise a single logical comparison, a complex numerical algorithm, or a rule-inference approach of an expert system. Output produced a 200-page report or may light a single LED. In effect, we can create a flow model for any computer-based system, regardless of size and complexity.

## Q.21) Explain different techniques in White Box Testing.

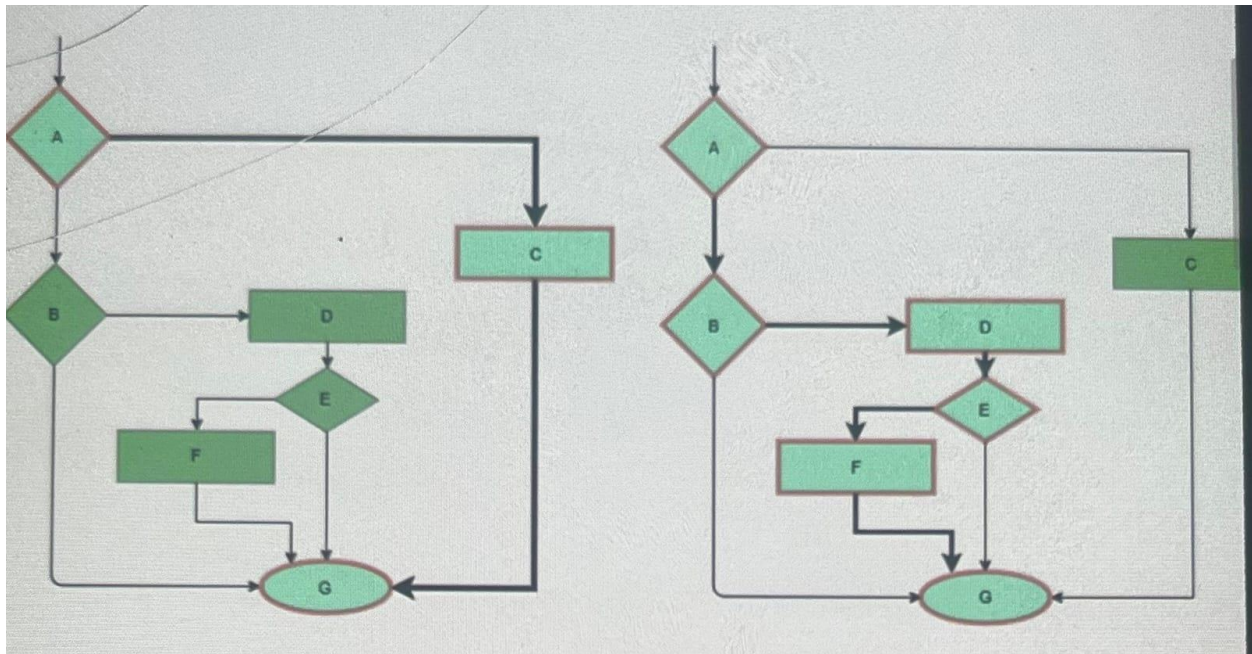
Ans:

### White Box Testing Techniques

One of the main benefits of white box testing is that it allows for testing every part of an application. To achieve complete code coverage, white box testing uses the following techniques:

1. Statement Coverage: In this technique, the aim is to traverse all statements at least once. Hence, each line of code is tested. In the case of a flowchart, every node must be traversed at least once. Since all lines of code are covered, it helps in pointing out faulty code.

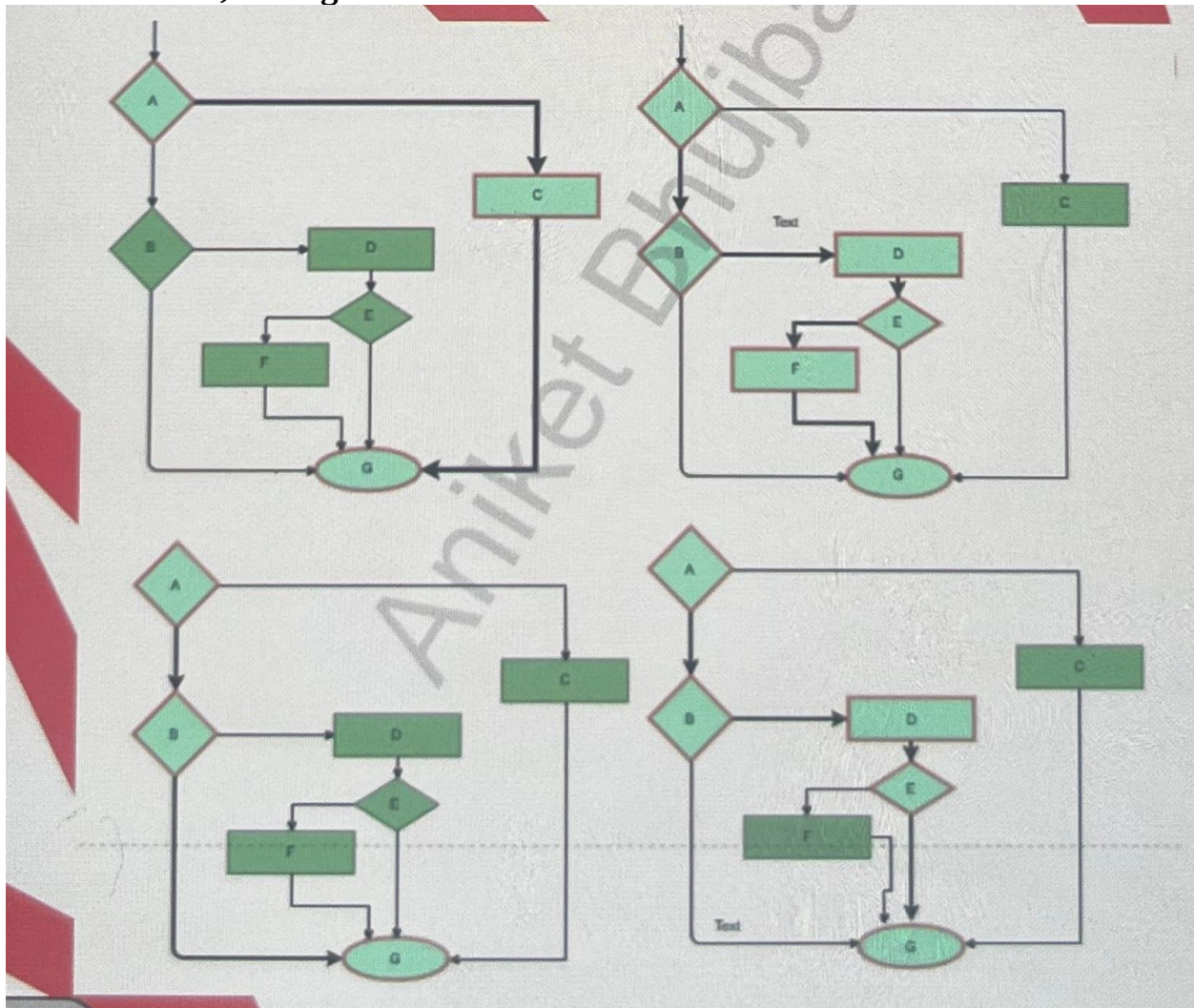
If we see in the case of a flowchart, every node must be traversed at least once. Since all lines of code are covered, it helps in pointing out faulty code while detecting.



2. Branch Coverage: Branch coverage focuses on testing the decision points or conditional branches in the code. It checks whether both possible outcomes (true and false) of each conditional statement are tested. In this technique, test cases are designed so that each branch from all decision points is traversed at least once. In a flowchart, all edges must be traversed at least once.



**In a flowchart, all edges must be traversed at least once**



### **Branch Coverage flowchart**

#### **3. Condition Coverage:**

In this technique, all individual conditions must be covered as shown in the following example:

READ X, Y

IF(X == 0 || Y == 0)

PRINT "0"

#TC1 – X = 0, Y = 55

#TC2 – X = 5, Y = 0

### 3. Multiple Condition Coverage:

In this technique, all the possible combinations of the possible outcomes of conditions are tested at least once. Let's consider the following example:

```
READ X, Y
IF(X == 0 || Y == 0)
PRINT "0"
#TC1: X = 0, Y = 0
#TC2: X = 0, Y = 5
#TC3: X = 55, Y = 0
#TC4: X = 55, Y = 5
```

### 5. Basis Path Testing:

In this technique, control flow graphs are made from code or flowchart and then Cyclomatic complexity is calculated which defines the number of independent paths so that the minimal number of test cases can be designed for each independent path.

#### Steps:

- Make the corresponding control flow graph
- Calculate the cyclomatic complexity
- Find the independent paths
- Design test cases corresponding to each independent path

$V(G) = P + 1$ , where P is the number of predicate nodes in the flow graph

$V(G) = E - N + 2$ , where E is the number of edges and N is the total number of nodes

$V(G)$  = Number of non-overlapping regions in the graph

#P1: 1 – 2 – 4 – 7 – 8

#P2: 1 – 2 – 3 – 5 – 7 – 8

#P3: 1 – 2 – 3 – 6 – 7 – 8

#P4: 1 – 2 – 4 – 7 – 1 – ... – 7 – 8

### 6. Loop Testing:

Loops are widely used and these are fundamental to many algorithms hence, their testing is very important. Errors often occur at the beginnings and ends of loops.

**1. Simple loops:** For simple loops of size  $n$ , test cases are designed that:



1. Skip the loop entirely
2. Only one pass through the loop
3. 2 passes
4.  $m$  passes, where  $m < n$
5.  $n-1$  and  $n+1$  passes

**2. Nested loops:** For nested loops, all the loops are set to their minimum count, and we start from the innermost loop. Simple loop tests are conducted for the innermost loop and this is worked outwards till all the loops have been tested.

**3. Concatenated loops:** Independent loops, one after another. Simple loop tests are applied for each. If they're not independent, treat them like nesting.

Q.22 Differentiate between the following:

*i) Scrum and Kanban*

| Scrum                                                                                                                                            | Kanban                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Scrum is an iterative and incremental Agile framework where work is divided into fixed time-boxed iterations called Sprints (usually 2–4 weeks). | Kanban is a continuous flow-based Agile method where work items move through stages on a visual board without fixed iterations. |
| It follows a structured process with predefined roles like Scrum Master, Product Owner, and Development Team.                                    | Kanban has no specific roles; existing team roles remain the same and focus is only on workflow visualization.                  |
| Scrum uses Sprint Planning, Daily Stand-ups, Sprint Review, and Retrospective as mandatory ceremonies.                                           | Kanban does not mandate any ceremonies; meetings are conducted as per team needs, making it more flexible.                      |
| In Scrum, scope is fixed during a sprint, and changes are discouraged                                                                            | In Kanban, changes can be made anytime since there are no sprint                                                                |

| Scrum                                                                                                                    | Kanban                                                                                                                             |
|--------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| until the sprint ends.                                                                                                   | boundaries.                                                                                                                        |
| Scrum limits work indirectly by fixing sprint duration and team capacity.                                                | Kanban explicitly uses Work-In-Progress (WIP) limits to manage task load and prevent overburdening.                                |
| Scrum requires estimation (story points) before starting each sprint.                                                    | Kanban focuses on measuring lead time and cycle time instead of estimations.                                                       |
| Scrum aims at achieving a potentially shippable product increment at the end of each sprint.                             | Kanban aims at continuous delivery by completing tasks when they are ready.                                                        |
| Scrum requires strict adherence to sprint goals, promoting predictability.                                               | Kanban promotes adaptability, allowing immediate response to priority changes.                                                     |
| Scrum performance is measured through velocity (stories per sprint).                                                     | Kanban performance is measured through cycle time and throughput.                                                                  |
| <b>Reason:</b> Scrum provides structure for teams new to Agile and ensures predictability through time-boxed iterations. | <b>Reason:</b> Kanban emphasizes flexibility and efficiency, ideal for teams needing continuous workflow without strict schedules. |

## ii) White Box Testing and Black Box Testing

| White Box Testing                                                                     | Black Box Testing                                                                                       |
|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| White Box Testing focuses on internal code structure, logic, and flow of the program. | Black Box Testing focuses on external functionality without considering the internal structure or code. |

| White Box Testing                                                                      | Black Box Testing                                                                                 |
|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Testers require knowledge of programming languages and to design test cases.           | Testers do not need programming knowledge; only functional requirements are needed.               |
| It helps in finding logical errors, loops, and paths.                                  | It helps in identifying functional errors, missing functions, and interface issues.               |
| Techniques include Statement Coverage, Branch Coverage, and Path Coverage.             | Techniques include Equivalence Partitioning, Boundary Value Analysis, and Decision Table Testing. |
| White Box Testing is usually performed by developers.                                  | Black Box Testing is usually performed by testers or QA teams.                                    |
| It is suitable for unit testing and verifying individual modules.                      | It is suitable for system and acceptance testing at the higher levels.                            |
| The main goal is to ensure that code implementation is correct and efficient.          | The main goal is to ensure that software behaves as expected for the user.                        |
| It is time-consuming due to in-depth code analysis.                                    | It is faster since it deals only with inputs and outputs.                                         |
| It can uncover hidden code errors or dead code.                                        | It can uncover functional defects or missing features.                                            |
| <b>Reason:</b> White Box is logic-based, ensuring the correctness of internal working. | <b>Reason:</b> Black Box is behaviour-based, ensuring the product meets user expectations.        |

### iii) FTR and Walkthrough

| Formal Technical Review (FTR)                                                                                      | Walkthrough                                                                                     |
|--------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| FTR is a structured, formal review process where a team of technical experts evaluates design, code, or documents. | A Walkthrough is a less formal review where the author presents the work to peers for feedback. |
| Conducted with defined roles like Moderator, Author, Recorder, and Reviewer.                                       | No formal roles are required; only the author and peers participate.                            |
| Focuses on detecting defects, logic errors, and compliance with standards.                                         | Focuses on understanding the content and gathering suggestions.                                 |
| Involves preparation and detailed documentation before the review meeting.                                         | Requires minimal preparation; often done in an informal meeting.                                |
| FTR outcomes are documented with action items and follow-ups.                                                      | Walkthrough outcomes are usually discussed verbally without formal records.                     |
| FTR helps in maintaining technical accuracy and consistency in large projects.                                     | Walkthrough helps in knowledge sharing and early feedback.                                      |
| Conducted at various development stages like design, code, and test plan review.                                   | Conducted early during document or design creation to improve clarity.                          |
| Time-consuming but ensures high defect detection rate.                                                             | Quick and interactive but may miss deeper technical issues.                                     |

| Formal Technical Review (FTR)                                        | Walkthrough                                                                                 |
|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| It requires strict entry and exit criteria.                          | It has no fixed criteria, making it more flexible.                                          |
| <b>Reason:</b> FTR ensures quality assurance and process compliance. | <b>Reason:</b> Walkthrough improves understanding and catches surface-level mistakes early. |

#### iv) Alpha and Beta Testing

| Alpha Testing                                                                         | Beta Testing                                                                        |
|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Alpha Testing is conducted by internal employees or testers within the organization.  | Beta Testing is conducted by actual users or customers in the real environment.     |
| It is performed at the developer's site before the product is released to the market. | It is performed at the user's site after Alpha Testing is completed.                |
| Conducted in a controlled environment with limited access.                            | Conducted in an uncontrolled environment with real-world usage.                     |
| The objective is to identify bugs before releasing the product to users.              | The objective is to get user feedback and identify usability or performance issues. |
| Testers have access to the source code and debugging tools.                           | Users do not have access to source code or debugging tools.                         |
| Issues found are immediately reported and fixed by the development team.              | Issues are reported by users, then analyzed and fixed in future releases.           |
| Alpha Testing includes both white                                                     | Beta Testing is purely black box in                                                 |

| Alpha Testing                                                                         | Beta Testing                                                                                                    |
|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| box and black box techniques.                                                         | nature.                                                                                                         |
| It is performed by QA teams and developers.                                           | It is performed by real customers or end users.                                                                 |
| Usually involves multiple test cycles before the public release.                      | Usually conducted once or twice before the final release.                                                       |
| <b>Reason:</b> Alpha Testing ensures product stability and quality before public use. | <b>Reason:</b> Beta Testing ensures customer satisfaction and verifies performance under real-world conditions. |

**Q.23) Write a short note on:**

**i) Boundary value analysis with suitable example.**

i) Boundary Value Analysis is a software testing technique used in Black Box Testing to identify errors at the boundaries (edges) of input domains rather than within the range.

ii) It is based on the principle that errors often occur at extreme ends or limits of input values.

iii) In most programs, the likelihood of defects is higher near the boundary values of the input rather than in the middle of the range.

iv) Therefore, BVA test cases are designed using boundary values of input ranges, such as the minimum, maximum, just below, and just above these limits.

v) The main goal of Boundary Value Analysis is to detect boundary-related defects and ensure that the system behaves correctly at its input limits.

vi) It helps in minimizing the number of test cases while maintaining effective coverage.

vii) Suppose a program accepts an input value for age between 18 and 60.

- Valid range:  $18 \leq \text{age} \leq 60$
- Test cases using BVA will be:
  - Just below lower boundary: 17 (Invalid)
  - Exact lower boundary: 18 (Valid)
  - Just above lower boundary: 19 (Valid)
  - Just below upper boundary: 59 (Valid)
  - Exact upper boundary: 60 (Valid)

- Just above upper boundary: 61 (Invalid)

viii) Boundary values are the most error-prone because programmers often make mistakes in defining limit conditions (e.g., using  $<$  instead of  $\leq$ ). Testing at these points ensures the software behaves correctly under edge conditions.

## ii) Six Sigma for software Engineering.

i) Six Sigma is a methodology for process improvement that helps organizations make their operations more efficient by identifying and removing errors and variations.

ii) Variations in processes cause errors, leading to defective products and poor customer satisfaction. Six Sigma aims to reduce such variations to lower costs and increase customer satisfaction.

iii) The concept of Six Sigma was introduced in 1986 by Bill Smith, an American engineer working at Motorola.

iv) It is widely used in manufacturing, service industries, government agencies, aerospace, and e-commerce to improve processes and product quality.

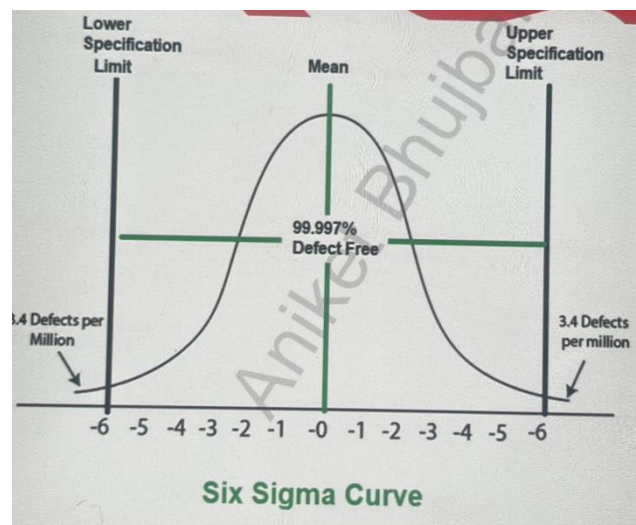
v) Six Sigma is both a methodology and a statistical concept that measures process variation and ensures high-quality output by identifying and eliminating causes of defects.

vi) A Six Sigma process has only 3.4 defects per million opportunities (is 99.99966% defect-free), showing near-perfect performance.

### vii) Characteristics:

- **Statistical Quality Control:** Derived from the Greek letter  $\sigma$  (sigma), meaning standard deviation, which measures process variation.
- **Methodical Approach:** Uses DMAIC (Define, Measure, Analyze, Improve, Control) and DMADV (Define, Measure, Analyze, Design, Verify) for improvement.
- **Fact and Data-Based:** Decisions are made through statistical and scientific analysis.
- **Objective and Project-Based:** Implemented with clear goals and measurable outcomes.
- **Customer Focus:** Quality standards are based on customer requirements and satisfaction.

- **Teamwork-Oriented:** Relies on collaboration and organized efforts within the organization for quality improvement.



**Q.24) Explain characteristics of SRS. Write an SRS for (Restaurant, hospital, library, medical management).**

**Ans: Characteristics of SRS (Software Requirements Specification)**

A good SRS clearly defines what the system should do and how it should perform under various conditions.

Below are the major characteristics explained in detail:

1. **Correctness**
  - The SRS must correctly represent all the requirements of the system.
  - *Example:* If a restaurant system must generate daily sales reports, this requirement must be clearly mentioned without missing details.
2. **Unambiguity**
  - Every requirement should have only one clear meaning, avoiding confusion.
  - *Example:* Instead of writing “fast response,” it should say “response time must be under 2 seconds.”
3. **Completeness**
  - The document must include all functional and non-functional requirements.
  - *Example:* A hospital system’s SRS should not miss features like patient registration or doctor assignment.
4. **Consistency**
  - Requirements should not conflict with each other.
  - *Example:* The SRS cannot say “system must allow only admins to add patients” and also “any user can add patients.”
5. **Verifiability**



- Each requirement should be testable through inspection, analysis, or testing.
  - *Example:* “System should support up to 100 users concurrently” can be tested.
  - 6. **Modifiability**
    - The SRS should be structured so that changes can be made easily without affecting the whole document.
    - *Example:* Updating “payment method” from cash to online should not require rewriting the entire document.
  - 7. **Traceability**
    - Each requirement should be uniquely identified so that it can be traced during development or testing.
    - *Example:* Each requirement is labeled as R1, R2, etc., and referred to during design and testing.
  - 8. **Design Independence**
    - The SRS should describe *what* the system should do, not *how* it will be implemented.
    - *Example:* It should say “system shall store user data,” not “use SQL database for storage.”
  - 9. **Feasibility**
    - The requirements must be realistic and achievable within time, cost, and technology limits.
    - *Example:* Asking a library system to “predict future book trends automatically” may not be feasible.
  - 10. **Conciseness and Understandability**
    - The SRS must be written clearly, without unnecessary jargon, and easy for both clients and developers to understand.
- 

## 1. SRS for Restaurant Management System

1. The system automates restaurant operations like table booking, order taking, billing, and report generation.
2. Users include waiters, customers, and managers.
3. Customers place orders digitally; orders go directly to the kitchen.
4. Bills are generated automatically after service completion.
5. Menu can be updated by admin anytime.
6. Touch-screen interface used by waiters for quick entry.
7. System connects through local Wi-Fi to sync all devices.
8. Response time must be under 2 seconds per operation.
9. Supports up to 50 simultaneous users.
10. Only managers can access revenue and stock reports.
11. Backup is taken automatically after each day’s closing.
12. Runs on Android tablets for easy mobility.

13. Maintains table-wise and order-wise tracking.
14. Must generate daily, weekly, and monthly sales reports.
15. Estimated duration for completion: 3 months.
16. Estimated budget: ₹1,50,000.

## 2. SRS for Hospital Management System

1. System automates hospital processes like patient registration, appointment scheduling, billing, and records maintenance.
2. Users include doctors, patients, receptionists, and administrators.
3. Patients can register online or at the counter.
4. Each patient is assigned a unique ID.
5. System stores personal details, diagnosis, and treatment history.
6. Doctors can update prescriptions and test reports.
7. Appointment schedules can be viewed and modified.
8. Bills are auto-generated after consultation or treatment.
9. Only authorized users can view or edit patient records.
10. Response time should not exceed 3 seconds for data retrieval.
11. Daily data backup is mandatory for security.
12. System supports 200 users at a time.
13. Accessible only on the hospital intranet for privacy.
14. System maintains logs for all operations performed.
15. Project duration: 4 months.
16. Budget estimated: ₹2,00,000.

## 3. SRS for Library Management System

1. System manages books, members, and transactions digitally.
2. Users include students, librarians, and administrators.
3. Every book and member gets a unique ID.
4. Books can be issued and returned with automatic due date calculation.
5. Overdue fines are calculated based on the number of late days.
6. System allows searching by title, author, or subject.
7. Barcode scanner integration for faster issue and return process.
8. Auto-generation of reports for issued and returned books.
9. Database can store 10,000+ books and 5,000 members.
10. Response time should be less than 2 seconds for queries.
11. Works on LAN with MySQL database support.
12. Automatic data backup daily to prevent data loss.
13. Only admin can add or remove books from inventory.
14. GUI must be simple and easy to use.
15. Project duration: 2.5 months.
16. Budget: ₹80,000.

#### 4. SRS for Medical Store Management System

1. System automates sales, purchase, billing, and stock updates of medicines.
2. Users include pharmacists, suppliers, and managers.
3. Each medicine has a code, name, price, and expiry date.
4. Stock quantity updates automatically after each sale.
5. Generates bills with item name, quantity, tax, and total amount.
6. Expiry alerts generated for medicines nearing expiration.
7. System maintains supplier details and purchase history.
8. Barcode scanner used for quick billing.
9. All data stored securely in database (SQLite).
10. Response time per operation under 2 seconds.
11. Handles up to 1 lakh transactions per month.
12. Auto-save every 5 minutes to prevent data loss.
13. Works on Windows OS.
14. User login authentication required for access.
15. Duration of project: 3 months.
16. Estimated budget: ₹1,20,000.